

PRACTICAL 7

Title

To Perform DOM Traversal and Updates – Develop a To-Do List App with Add, Edit and Delete Features

Aim

To perform DOM traversal and updates using JavaScript by developing a To-Do List application with Add, Edit, and Delete features, and to develop a Student Registration Portal with form validations.

Objectives

- To understand DOM manipulation.
- To add, edit, and delete HTML elements dynamically.
- To validate user input using JavaScript.
- To implement email and mobile validation.
- To validate student registration details.

EXPERIMENT

Problem Statement

Develop a To-Do List application using HTML, CSS, and JavaScript with the following features:

- Add a task.
- Edit a task.
- Delete a task.
- Dynamically update the task list using DOM methods.

Files Required

```
Practical7/  
├── index.html  
├── style.css  
└── script.js
```

1. index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">
```

```

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>To-Do List</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

<div class="container">

    <h1>To-Do List</h1>

    <div class="input-area">
        <input type="text" id="taskInput"
            placeholder="Enter a task">

        <button onclick="addTask()">Add</button>
    </div>

    <ul id="taskList"></ul>

</div>

<script src="script.js"></script>

</body>
</html>

```

2. style.css

```

* {
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background: #f2f2f2;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}

.container {
    width: 500px;
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 5px 15px #bbb;
}

h1 {
    text-align: center;
}

.input-area {
    display: flex;
    gap: 10px;
    margin: 20px 0;
}

```

```

input {
  flex: 1;
  padding: 10px;
}

button {
  padding: 10px 15px;
  border: none;
  border-radius: 5px;
  background: #007bff;
  color: white;
  cursor: pointer;
}

li {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 10px;
  margin: 8px 0;
  background: #f5f5f5;
  border-radius: 5px;
}

.task-text {
  flex: 1;
}

.edit-btn {
  background: green;
}

.delete-btn {
  background: red;
}

```

3. script.js

```

function addTask() {

  const input = document.getElementById("taskInput");
  const task = input.value.trim();

  if (task === "") {
    alert("Please enter a task.");
    return;
  }

  const li = document.createElement("li");

  const span = document.createElement("span");
  span.textContent = task;
  span.className = "task-text";

  const edit = document.createElement("button");
  edit.textContent = "Edit";
  edit.className = "edit-btn";

  edit.onclick = function () {
    const newTask = prompt("Edit task:", span.textContent);

```

```

        if (newTask && newTask.trim() !== "") {
            span.textContent = newTask.trim();
        }
    };

    const del = document.createElement("button");
    del.textContent = "Delete";
    del.className = "delete-btn";

    del.onclick = function () {
        if (confirm("Delete this task?")) {
            li.remove();
        }
    };

    li.appendChild(span);
    li.appendChild(edit);
    li.appendChild(del);

    document.getElementById("taskList").appendChild(li);

    input.value = "";
}

```

Working

The application uses DOM methods such as:

```

document.getElementById()
document.createElement()
appendChild()
remove()
textContent

```

`createElement()` creates new elements, `appendChild()` adds them to the page, and `remove()` deletes a selected task.

CASE STUDY

Student Registration Portal

Aim

To develop a Student Registration Portal with validations for student details including Student ID, email, mobile number, skills, branch, city, resume upload, and Terms and Conditions.

Form Fields

- Student ID
- Student Name

- Email
- Mobile Number
- Skills
- Branch
- City
- Resume Upload
- Terms and Conditions
- Register Button

Registration HTML

```
<form onsubmit="return validateForm()">

    <input type="text" id="studentId"
        placeholder="Student ID">

    <input type="text" id="studentName"
        placeholder="Student Name">

    <input type="text" id="email"
        placeholder="Email">

    <input type="text" id="mobile"
        placeholder="Mobile Number"
        maxlength="10">

    <p>Skills:</p>

    <label>
        <input type="checkbox" name="skills" value="HTML">
        HTML
    </label>

    <label>
        <input type="checkbox" name="skills" value="CSS">
        CSS
    </label>

    <label>
        <input type="checkbox" name="skills" value="JavaScript">
        JavaScript
    </label>

    <select id="branch">
        <option value="">Select Branch</option>
        <option value="CSE">CSE</option>
        <option value="IT">IT</option>
        <option value="ECE">ECE</option>
    </select>

    <select id="city">
        <option value="">Select City</option>
        <option value="Nagpur">Nagpur</option>
        <option value="Pune">Pune</option>
        <option value="Mumbai">Mumbai</option>
    </select>

    <input type="file" id="resume">
```

```

        accept=".pdf,.doc,.docx">

<label>
    <input type="checkbox" id="terms">
    I agree to Terms and Conditions
</label>

<button type="submit">Register</button>

</form>

```

Registration JavaScript

```

function validateForm() {

    const studentId =
        document.getElementById("studentId").value.trim();

    const name =
        document.getElementById("studentName").value.trim();

    const email =
        document.getElementById("email").value.trim();

    const mobile =
        document.getElementById("mobile").value.trim();

    const branch =
        document.getElementById("branch").value;

    const city =
        document.getElementById("city").value;

    const resume =
        document.getElementById("resume").files[0];

    const terms =
        document.getElementById("terms").checked;

    if (studentId === "") {
        alert("Please enter Student ID.");
        return false;
    }

    if (name === "") {
        alert("Please enter your name.");
        return false;
    }

    const emailPattern =
        /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (!emailPattern.test(email)) {
        alert("Enter a valid email.");
        return false;
    }

    const mobilePattern = /^[0-9]{10}$/;

    if (!mobilePattern.test(mobile)) {

```

```

        alert("Enter a valid 10-digit mobile number.");
        return false;
    }

    const skills =
        document.querySelectorAll(
            'input[name="skills"]:checked'
        );

    if (skills.length === 0) {
        alert("Select at least one skill.");
        return false;
    }

    if (branch === "") {
        alert("Select your branch.");
        return false;
    }

    if (city === "") {
        alert("Select your city.");
        return false;
    }

    if (!resume) {
        alert("Please upload your resume.");
        return false;
    }

    if (!terms) {
        alert("Please agree to Terms and Conditions.");
        return false;
    }

    alert("Registration successful!");

    return false;
}

```

Validations Used

Field	Validation
Student ID	Cannot be empty
Name	Cannot be empty
Email	Valid email format
Mobile	Exactly 10 digits
Skills	At least one skill
Branch	Must be selected
City	Must be selected
Resume	Must be uploaded
Terms	Must be accepted

Result

The To-Do List application was successfully developed with **Add, Edit, and Delete** functionality using DOM manipulation.

The Student Registration Portal was successfully developed with multiple validations for student details, email, mobile number, skills, branch, city, resume, and Terms and Conditions.

Conclusion

This practical demonstrates **DOM traversal, DOM updates, dynamic element creation, event handling, and JavaScript form validation** through a To-Do List application and a Student Registration Portal.